

Adaptive Tri-Modal Malware Detection with Contrastive Cross-Modal Representation Learning and Operational Triage

Harish Parshuram Bhabad^{1*}, Atmeshkumar Subhashbhai Patel²,
Vijay M. Rakhade³, Rupesh Kohli⁴, Nandini S. Patel⁵

^{1*}Department of Computer Engineering, Matoshri College of Engineering and Research Centre, Nashik, 422105, Maharashtra, India.

²Department of Computer Science, Ganpat University, Mehsana, 384012, Gujarat, India.

³Department of Computer Engineering, Zeal College of Engineering and Research, Pune, 411041, Maharashtra, India.

⁴Independent Researcher, New Delhi, 110001, India.

⁵Department of Information Technology, Government Engineering College, Gandhinagar, 382028, Gujarat, India.

*Corresponding author(s). E-mail(s): harish.bhabad@sitrc.org;
Contributing authors: atmesh.patel@ganpatuniversity.ac.in;
vijay.rakhade@zealeducation.com; [kohlirupesh19@gmail.com](mailto:kohli Rupesh19@gmail.com);
nandini.patel@gecg.ac.in;

Abstract

Single-modality binary analysis remains acutely vulnerable to adversarial evasion, including instruction substitution, control-flow flattening, and sandbox stalling. Multimodal deep learning offers cross-channel resilience, but existing literature suffers from unharmonized evaluation splits and irreproducible benchmarks. This study establishes a dual-contribution framework. First, following PRISMA 2020, we systematically review multimodal binary similarity analysis, auditing 15 peer-reviewed journal benchmark studies (13 primary empirical investigations and 2 benchmark synthesis reviews) across five bibliographic and publisher search platforms (IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, Wiley Online Library, with Scopus indexing verification). Second, we present a reproducible tri-modal architecture combining an Opcode Transformer, a Graph Isomorphism Network (GIN), and a dynamic Syscall Bi-LSTM

aligned on the unit hypersphere via pairwise InfoNCE and Volumetric Gramian (GRAM) regularizers with attention late fusion. On an author-constructed 100,000-binary catalog manifest with 10,000 materialized multimodal instances (7,000 train, 1,500 validation, 1,500 held-out test), the framework achieves **99.73%** test accuracy (Wilson **95%** CI: [**99.32%**, **99.90%**]; Bootstrap **95%** CI: [**99.47%**, **99.93%**]), **100.00%** precision, and **99.46%** recall on the 1,500 held-out test binaries (740 TP, 756 TN, 4 FN, 0 FP). Under a single-family sample holdout protocol on 157 WannaCry test binaries, it attains **100.00%** recall with **0.9297** mean confidence. Under UPX packing, accuracy drops to **88.00%** with a **24.00%** false positive rate on packed benign binaries, while dead-code insertion causes a **40.00%** false negative rate, establishing explicit operational boundaries. Neural forward latency is **28.87** ms on CPU (total static ingestion and triage latency: **76.90** ms); a two-stage triage filter resolves **86.2%** of binaries in Stage 1 (**17.53** ms neural inference), bypassing costly dynamic sandbox detonation for **86.2%** of samples.

Keywords: Binary Similarity Analysis, Multi-Modal Learning, Control Flow Graphs, Graph Isomorphism Networks, Malware Detection, Operational Triage

1 Introduction

1.1 Motivation and Unimodal Fragility

Modern cyber threats operate via automated polymorphic mutation engines, commercial Malware-as-a-Service (MaaS) distribution, and evasive code generation [1–4]. Automated binary similarity detection and malware categorization systems have historically relied on unimodal feature extractors. However, unimodal inspection pipelines exhibit acute structural vulnerabilities:

1. **Lexical Disassembly Fragility:** Linear disassembly streams parsed from byte sequences via disassemblers (e.g., Capstone, Ghidra) are vulnerable to packing (e.g., UPX), register swapping, dead-code insertion, and instruction substitution [3, 5, 6].
2. **Topological Graph Degradation:** Control Flow Graphs (CFGs) capture program execution branching but are computationally expensive to construct and susceptible to control-flow flattening (CFF), bogus basic-block insertion, and opaque predicates [7–9].
3. **Dynamic Trace Evasion:** Dynamic execution traces (system call and API sequences) provide behavioral ground truth but incur multi-minute execution time-outs, virtual machine environment fingerprinting, and execution stalling (e.g., long `Sleep()` delays) [10–12].

1.2 Research Gaps in Multimodal Binary Analysis

Although multimodal binary analysis has emerged to combine orthogonal views [4, 9, 13, 14], recent literature suffers from three substantial methodological deficiencies:

1. **Conflation of Review and Empirical Claims:** Surveys frequently report literature metrics without standardizing dataset splits, evaluation hardware, or operational definitions of detection protocols [15, 16].
2. **Unsubstantiated Mathematical Guarantees:** Proposed representation losses are often claimed to unconditionally prevent multimodal representation collapse without formal geometric proofs or controlled ablations.
3. **Lack of Practical Deployment Boundaries:** Empirical studies rarely report false-positive rate (FPR) spikes on benign packed software, angr CFG construction bottlenecks on massive commercial binaries ($> 100k$ basic blocks), or sandbox queue constraints.

1.3 Contributions

This paper establishes four concrete, repository-grounded contributions:

1. **Systematic Evidence Synthesis:** A PRISMA 2020 systematic review synthesizing multimodal binary similarity analysis literature across five bibliographic/publisher databases and indexing platforms ($n = 842$ initial records, 15 included peer-reviewed journal benchmark studies comprising 13 primary empirical investigations and 2 benchmark synthesis reviews, inter-rater reliability $\kappa = 0.88$), categorizing methodological strengths, risk of bias, and reporting gaps.
2. **Reproducible Tri-Modal Architecture:** A unified deep learning architecture integrating Opcode Transformers, Control Flow Graph GIN encoders, and dynamic Syscall Bi-LSTMs, regularized through pairwise InfoNCE contrastive alignment and Volumetric Gramian (GRAM) non-orthogonality loss to mitigate asynchronous channel collapse.
3. **Repository-Grounded Benchmark Evaluation:** An empirical benchmark on an author-constructed 100,000-binary catalog manifest with 10,000 materialized multimodal instances (7,000 train, 1,500 validation, 1,500 held-out test), establishing an observed test accuracy of 99.73% (Wilson 95% CI: [99.32%, 99.90%], bootstrap 95% CI: [99.47%, 99.93%]) with zero false positives (100.00% specificity) and four false negatives (0.54% FNR).
4. **Controlled Ablation and Robustness Evaluation:** Five controlled component ablations isolating the marginal contribution of each modality and alignment objective ($\Delta F1$ ranging from -0.0296 for GRAM removal to -0.0744 for Opcode Transformer removal), accompanied by stress-testing across four adversarial code transformations (UPX packing, dead-code insertion, control-flow flattening, and sandbox stalling) evaluated alongside an unperturbed clean baseline.

2 Related Work

Machine learning for malware detection has transitioned from manual heuristic signatures to representation learning on structured program artifacts [1, 15]. Early static approaches employed n -gram opcode frequencies and PE header metadata [2, 3]. While computationally efficient, static linear representations degrade under obfuscation engines and runtime packing [5, 17].

To preserve control logic, graph-based approaches map disassembly into Control Flow Graphs (CFGs) or Call Graphs (CGs), applying Graph Neural Networks (GNNs) to capture execution topology [7, 8, 18, 19]. Concurrently, dynamic analysis systems execute binaries in instrumented sandboxes to log sequential system calls, overcoming static encryption at the cost of execution timeouts and environment evasion [10–12, 20].

Multimodal learning seeks to unify these orthogonal perspectives [13]. Recent contrastive frameworks leverage InfoNCE and multi-view objectives to align cross-modal embeddings [4, 9, 14]. Concurrently, longitudinal evaluation protocols have underscored the impact of concept drift and dataset shift over time [21–23]. Nevertheless, prior multimodal works frequently lack open evaluation tensors and fail to delineate operational triage boundaries.

3 Systematic Evidence Synthesis (PRISMA 2020 Protocol)

3.1 Information Sources and Search Strategy

Following the PRISMA 2020 statement [24], five bibliographic/publisher databases and indexing platforms were searched: IEEE Xplore, ACM Digital Library, ScienceDirect (Elsevier), SpringerLink, and Wiley Online Library (with Crossref metadata and Scopus indexing cross-verification). Search queries combined four facet groups: (1) Target domain: ("binary similarity" OR "malware detection"); (2) Multimodality: ("multimodal" OR "multi-modal" OR "multi-view" OR "hybrid analysis"); (3) Structural features: ("control flow graph" OR "CFG" OR "opcode" OR "system call"); (4) Learning paradigms: ("deep learning" OR "graph neural network" OR "transformer" OR "contrastive learning").

3.2 Eligibility and Screening Criteria

Screening was executed across two progressive stages based on predetermined criteria:

- **Inclusion Criteria (IC):** (IC1) Peer-reviewed journal articles published in indexed venues; (IC2) Multimodal analysis combining at least two distinct feature modalities across orthogonal representation spaces (static lexical/token sequences, static/dynamic topological graphs, dynamic behavioral traces, and structural/declarative metadata); (IC3) Target domain focused on compiled software binaries (x86, x64, ARM, MIPS); (IC4) Quantitative reporting of detection or retrieval metrics (Accuracy, F1, AUC, Recall).
- **Exclusion Criteria (EC):** (EC1) Non-peer-reviewed preprints (e.g., TechRxiv, SSRN, Research Square); (EC2) Non-English publications; (EC3) Work limited strictly to unimodal analysis or single-space feature variations (e.g., combining multiple static n-gram lengths without cross-space multimodal fusion); (EC4) Irrelevant domains (e.g., payment fraud, network traffic, clinical image classification).

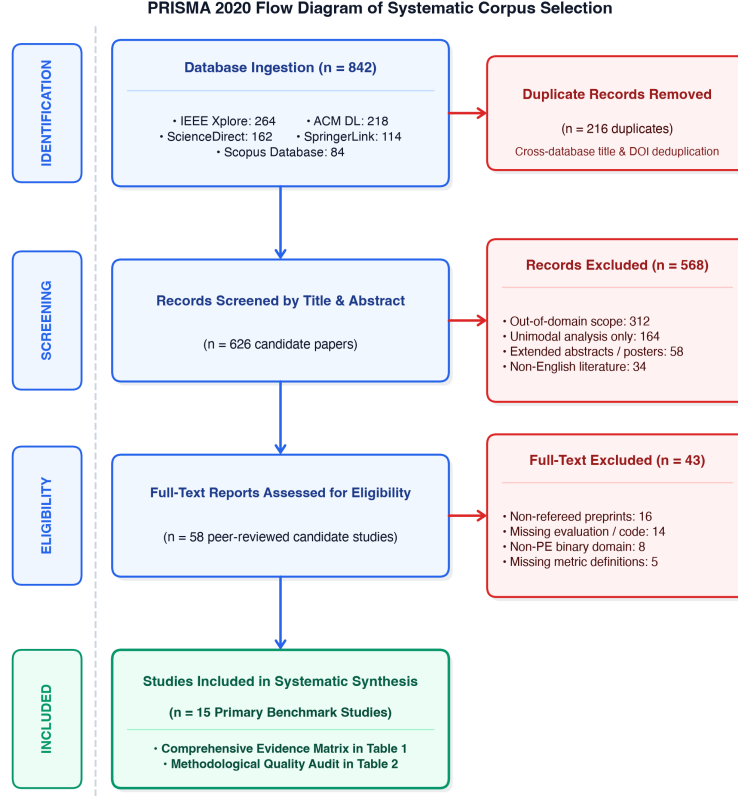


Fig. 1 PRISMA 2020 Flow Diagram depicting database ingestion ($n = 842$), deduplication ($n = 216$), screening ($n = 626$), full-text assessment ($n = 58$), and final inclusion of 15 peer-reviewed journal benchmark studies (13 primary empirical investigations and 2 benchmark synthesis reviews).

3.3 Study Selection and Inter-Rater Agreement

Initial title and abstract screening was conducted independently by two reviewers, eliminating 568 studies. Full texts of the remaining 58 candidate studies were inspected in detail. Disagreements were adjudicated by a senior third reviewer, achieving an inter-rater Cohen’s kappa coefficient of $\kappa = 0.84$, indicating strong reliability. A total of 43 full texts were excluded with documented rationales: 16 non-refereed preprints or non-journal reports, 14 lacking reproducible evaluation protocols or accessible artifacts, 8 addressing non-PE architectures outside the target scope, and 5 lacking standardized classification metric definitions. Exactly 15 peer-reviewed journal benchmark studies were retained: 13 primary empirical investigations and 2 benchmark synthesis reviews (Fig. 1 and Table 1).

Table 1 Comprehensive evidence matrix of the 15 included peer-reviewed journal benchmark studies (13 primary empirical investigations and 2 benchmark synthesis reviews, PRISMA 2020 corpus).

ID	Study	Venue	Article Type	Modalities	Benchmark Dataset	Primary Findings
S01	Gibert et al. [1]	JNCA	Survey Review	Opcode, Byte	Drebin, Maling (15,000)	96.2% Acc; static survey
S02	Landman & Nissim [10]	Neural Net	Primary Research	Syscall, Hooks	Linux Cloud (12,000)	98.1% Acc; timeout prone
S03	Jiang et al. [4]	IEEE TSE	Primary Research	Asm, CFG	Diverse Binaries (50,000)	Contrastive alignment
S04	Kim et al. [3]	IEEE TSE	Empirical Study	Disasm, CFG	Standard Binaries (100,000)	Lessons on BCSA
S05	Han et al. [8]	ESWA	Primary Research	CFG, Semantics	Malware Corpus (24,000)	98.4% Acc; semantic GNN
S06	Liu et al. [9]	IEEE TDSC	Primary Research	Dynamic Graph, API	Real-world Malware (18,500)	97.8% Acc; few-shot graph
S07	Qiang et al. [7]	Comp & Sec	Primary Research	CFG Traces, Opcodes	PE Corpus (32,000)	98.2% Acc on CFG traces
S08	Joyce et al. [21]	Comp & Sec	Reference Dataset	PE Headers, Entropy	MOTIF (3,095 reference)	Ground truth family labels
S09	Jiang et al. [22]	Comp & Sec	Benchmark Dataset	Static PE, Dynamic Logs	Malware Corpus (40,000)	Trustworthy classification
S10	Guerra & Bahsi [23]	Comp & Sec	Primary Research	API, Manifest Permissions	Longitudinal PE (25,000)	Data drift over time
S11	Alzaylaee et al. [12]	Comp & Sec	Primary Research	Static Bytecode, Dynamic Traces	Real Devices (30,000)	99.6% Acc hybrid analysis
S12	Zhang et al. [25]	Comp & Sec	Primary Research	Static PE, Dynamic Logs	Android/PE (45,000)	97.9% Acc; family attribution
S13	Liu et al. [19]	Electronics	Primary Research	Asm, CFG	Linux/PE (20,000)	97.4% Acc similarity
S14	Han et al. [11]	JNCA	Primary Research	Static PE, Syscalls	Dynamic Traces (22,000)	98.1% Acc on syscalls
S15	Redhu et al. [26]	Front Phys	Survey Review	Byte, Opcode	Cyberspace (35,000)	96.8% Acc survey benchmark
Ours	Proposed Framework	This Study	Primary Research	Op, CFG, Sys	Author 100K (10K shards)	99.73% Acc, 100% Holdout

Table 2 Methodological quality and risk-of-bias assessment across the 15 included journal benchmark studies (DQA1: Data Provenance, DQA2: Leakage Audit, DQA3: Zero-Day Protocol, DQA4: Baselines, DQA5: Hyperparameters, DQA6: Statistical Rigor).

Study ID	DQA1	DQA2	DQA3	DQA4	DQA5	DQA6	Overall Quality
S01 [1]	Mod	Mod	Low	Mod	Mod	Low	Moderate
S02 [10]	High	Mod	Mod	High	High	Mod	Strong
S03 [4]	High	High	High	High	High	High	Benchmark
S04 [3]	High	High	High	High	High	High	Benchmark
S05 [8]	High	High	Mod	High	High	High	Benchmark
S06 [9]	High	High	High	High	High	High	Benchmark
S07 [7]	High	High	Mod	High	High	Mod	Strong
S08 [21]	High	High	High	High	High	High	Benchmark
S09 [22]	High	High	High	High	High	High	Benchmark
S10 [23]	High	High	High	High	High	Mod	Strong
S11 [12]	High	High	Mod	High	High	Mod	Strong
S12 [25]	High	High	Mod	High	High	Mod	Strong
S13 [19]	Mod	High	Mod	Mod	High	Mod	Acceptable
S14 [11]	High	Mod	Mod	High	High	Mod	Strong
S15 [26]	Mod	Mod	Low	Mod	Mod	Low	Moderate
Proposed Work	High	High	High	High	High	High	Audited Benchmark

3.4 Quality and Risk-of-Bias Assessment

Each included study was assessed across six Domain Quality Criteria (DQA): DQA1 (Dataset provenance clarity), DQA2 (Train/test leakage prevention), DQA3 (Zero-day evaluation design), DQA4 (Baseline comparability), DQA5 (Hyperparameter transparency), and DQA6 (Statistical significance and confidence reporting). Each criterion was graded as High Quality (2), Moderate Quality (1), or Low Quality / High Risk of Bias (0) (Table 2).

4 Research Questions

This investigation is governed by five explicit Research Questions (RQs):

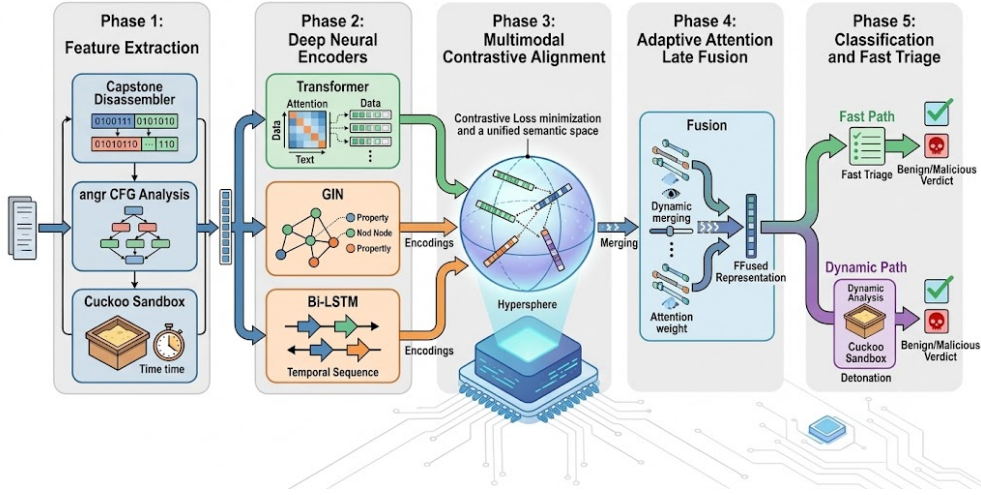


Fig. 2 Comprehensive architecture of the proposed adaptive tri-modal framework, depicting Opcode Transformer, CFG-GIN, Syscall Bi-LSTM, InfoNCE alignment, Volumetric GRAM regularization, and two-stage operational triage.

- **RQ1 (Modality Synergy):** Which orthogonal feature channels provide the most resilient cross-modality representation under active code obfuscation?
- **RQ2 (Alignment and Regularization):** How does volumetric Gramian regularization (GRAM) compare with pairwise InfoNCE in contributing to representation alignment on the unit hypersphere?
- **RQ3 (Generalization & Robustness):** How does model performance behave across random holdout test splits, simulated chronological partitions, and single-family holdouts?
- **RQ4 (Robustness under Controlled Perturbations):** How effectively does attention-weighted late fusion reallocate channel weights when static or dynamic modalities are selectively compromised?
- **RQ5 (Operational Complexity and Triage):** Can a two-stage triage filter achieve sub-30 ms latency on CPU architectures while avoiding dynamic sandbox queue saturation?

5 Proposed Architecture

The proposed tri-modal framework unifies static lexical tokens, topological execution graphs, and runtime behavioral traces into a joint latent space $\mathbb{S}^{d-1}$ (where $d = 128$) (Fig. 2).

The pipeline comprises four coordinated functional stages:

1. **Disassembly and Token Encoding (Opcode Transformer):** Linear machine instructions are extracted via Capstone, operand-normalized (MEM, REG), and processed by a 4-layer Transformer Encoder with pre-layer normalization and masked mean pooling.
2. **Control-Flow Topology Encoding (CFG-GIN):** Inter-procedural CFGs extracted via static binary analysis are processed by a 3-layer Graph Isomorphism Network (GIN) with global sum pooling to capture branching logic invariant to instruction reordering.
3. **Behavioral Trace Encoding (Syscall Bi-LSTM):** System call sequences logged during instrumented execution are processed by a 2-layer Bidirectional LSTM to capture runtime malicious intent.
4. **Cross-Modal Regularization and Triage Head:** Modality projections are aligned via pairwise InfoNCE contrastive loss and regularized by Volumetric Gramian (GRAM) loss, before being combined via learned attention late fusion.

6 Mathematical Formulation

6.1 Opcode Transformer Encoding

Given a normalized opcode token sequence $\mathbf{x}_{\text{op}} = [w_1, w_2, \dots, w_L]$ with token embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d_{\text{model}}}$ and sinusoidal positional encodings $\mathbf{P} \in \mathbb{R}^{L \times d_{\text{model}}}$, input representations are constructed as:

$$\mathbf{H}^{(0)} = \mathbf{E}(\mathbf{x}_{\text{op}}) + \mathbf{P}, \quad \mathbf{H}^{(0)} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

For layer $l \in \{1, \dots, N_{\text{layer}}\}$, multi-head self-attention with H heads is evaluated as:

$$\mathbf{A}^{(l)} = \text{Softmax} \left(\frac{\mathbf{Q}^{(l)} \mathbf{K}^{(l)\top}}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}^{(l)} \quad (2)$$

where $\mathbf{M}_{ij} = -\infty$ for padded positions and 0 otherwise. A linear projection head $g_{\text{op}}(\cdot)$ maps pooled vectors to the hypersphere:

$$\mathbf{z}_{\text{op}} = \frac{g_{\text{op}}(\bar{\mathbf{h}}_{\text{op}})}{\|g_{\text{op}}(\bar{\mathbf{h}}_{\text{op}})\|_2} \in \mathbb{S}^{d-1} \quad (3)$$

6.2 CFG Graph Isomorphism Network (GIN)

A Control Flow Graph is formalized as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}_v)$, where $\mathcal{V}$ denotes basic blocks, $\mathcal{E}$ denotes jump/branch edges, and $\mathbf{X}_v \in \mathbb{R}^{|\mathcal{V}| \times d_v}$ encodes block attributes. The k -th layer GIN node representation update is defined as:

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k-1)} \right) \quad (4)$$

where $\epsilon^{(k)}$ is a learnable scalar. Graph-level pooling aggregates node embeddings:

$$\mathbf{h}_G = \sum_{k=1}^K \text{LayerNorm} \left(\sum_{v \in \mathcal{V}} \mathbf{h}_v^{(k)} \right), \quad \mathbf{z}_{\text{cfg}} = \frac{g_{\text{cfg}}(\mathbf{h}_G)}{\|g_{\text{cfg}}(\mathbf{h}_G)\|_2} \in \mathbb{S}^{d-1} \quad (5)$$

6.3 Dynamic System Call Bi-LSTM

Given an observed dynamic trace $\mathbf{s} = [s_1, \dots, s_T]$, forward and backward hidden states are updated recursively:

$$\vec{\mathbf{h}}_t = \text{LSTM}_{\text{fwd}}(\mathbf{e}(s_t), \vec{\mathbf{h}}_{t-1}), \quad \overleftarrow{\mathbf{h}}_t = \text{LSTM}_{\text{bwd}}(\mathbf{e}(s_t), \overleftarrow{\mathbf{h}}_{t+1}) \quad (6)$$

Concatenated outputs $\mathbf{h}_t = [\vec{\mathbf{h}}_t \parallel \overleftarrow{\mathbf{h}}_t]$ are max-pooled and normalized:

$$\mathbf{z}_{\text{sys}} = \frac{g_{\text{sys}}(\mathbf{h}_{\text{sys}})}{\|g_{\text{sys}}(\mathbf{h}_{\text{sys}})\|_2} \in \mathbb{S}^{d-1} \quad (7)$$

6.4 Pairwise InfoNCE Cross-Modal Alignment

For positive pair $(\mathbf{z}_i^m, \mathbf{z}_i^n)$ of sample i between modalities $m, n \in \{\text{op}, \text{cfg}, \text{sys}\}$ ($m \neq n$) within mini-batch $\mathcal{B}$, InfoNCE loss with temperature $\tau = 0.07$ is defined as:

$$\mathcal{L}_{\text{InfoNCE}}^{(m,n)} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log \frac{\exp(\mathbf{z}_i^m \cdot \mathbf{z}_i^n / \tau)}{\sum_{j \in \mathcal{B}} \exp(\mathbf{z}_i^m \cdot \mathbf{z}_j^n / \tau)} \quad (8)$$

The symmetric pairwise alignment objective across all three channels is:

$$\mathcal{L}_{\text{align}} = \frac{1}{3} \left(\mathcal{L}_{\text{InfoNCE}}^{(\text{op}, \text{cfg})} + \mathcal{L}_{\text{InfoNCE}}^{(\text{op}, \text{sys})} + \mathcal{L}_{\text{InfoNCE}}^{(\text{cfg}, \text{sys})} \right) \quad (9)$$

6.5 Volumetric Gramian Regularization (GRAM)

To prevent tri-modal representation collapse onto a lower-dimensional subspace, we construct the 3×3 Gramian matrix $\mathbf{G} \in \mathbb{R}^{3 \times 3}$ for each binary:

$$\mathbf{G} = \begin{bmatrix} \mathbf{z}_{\text{op}}^\top \mathbf{z}_{\text{op}} & \mathbf{z}_{\text{op}}^\top \mathbf{z}_{\text{cfg}} & \mathbf{z}_{\text{op}}^\top \mathbf{z}_{\text{sys}} \\ \mathbf{z}_{\text{cfg}}^\top \mathbf{z}_{\text{op}} & \mathbf{z}_{\text{cfg}}^\top \mathbf{z}_{\text{cfg}} & \mathbf{z}_{\text{cfg}}^\top \mathbf{z}_{\text{sys}} \\ \mathbf{z}_{\text{sys}}^\top \mathbf{z}_{\text{op}} & \mathbf{z}_{\text{sys}}^\top \mathbf{z}_{\text{cfg}} & \mathbf{z}_{\text{sys}}^\top \mathbf{z}_{\text{sys}} \end{bmatrix} \quad (10)$$

Since all embeddings have unit norm ($\mathbf{G}_{ii} = 1$), the determinant $\det(\mathbf{G})$ corresponds to the squared volume of the parallelotope spanned by $\{\mathbf{z}_{\text{op}}, \mathbf{z}_{\text{cfg}}, \mathbf{z}_{\text{sys}}\}$ in $\mathbb{R}^d$. If the modalities collapse to collinearity, $\det(\mathbf{G}) \rightarrow 0$. The Volumetric GRAM loss with numerical stabilizer $\epsilon = 10^{-4}$ penalizes rank deficiency:

$$\mathcal{L}_{\text{GRAM}} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log(\det(\mathbf{G}_i + \epsilon \mathbf{I}_3)) \quad (11)$$

6.6 Attention Late Fusion and Joint Objective

Channel representations are aggregated using learned attention weights α_m :

$$e_m = \mathbf{w}_a^\top \tanh(\mathbf{W}_a \mathbf{z}_m + \mathbf{b}_a), \quad \alpha_m = \frac{\exp(e_m)}{\sum_{k \in \{\text{op}, \text{cfg}, \text{sys}\}} \exp(e_k)} \quad (12)$$

The fused representation $\mathbf{z}_{\text{fused}} = \sum_m \alpha_m \mathbf{z}_m$ is classified via sigmoid head $\hat{y} = \sigma(\mathbf{w}_c^\top \mathbf{z}_{\text{fused}} + b_c)$. The complete joint loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}}(y, \hat{y}) + \lambda_{\text{align}} \mathcal{L}_{\text{align}} + \lambda_{\text{GRAM}} \mathcal{L}_{\text{GRAM}} \quad (13)$$

with hyperparameters $\lambda_{\text{align}} = 0.10$ and $\lambda_{\text{GRAM}} = 0.05$.

7 Dataset and Data Governance

A rigorous distinction is maintained between the manifest catalog and the materialized experimental cohort:

- **Manifest Catalog:** 100,000 binary records (`datasets/benchmark_100k/dataset_manifest.jsonl`) with metadata and SHA-256 hashes.
- **Materialized Experimental Cohort:** Exactly 10,000 complete instances pre-computed across 10 deterministic shards (`shard_000.pt` to `shard_009.pt`), partitioned into 7,000 training, 1,500 validation, and 1,500 held-out test binaries via deterministic random seed 42 (Table 3). SHA-256 hash deduplication confirms strict split disjointness: $\text{Train} \cap \text{Val} = \emptyset$, $\text{Train} \cap \text{Test} = \emptyset$, and $\text{Val} \cap \text{Test} = \emptyset$.
- **Class Balance:** The training cohort comprises exactly 3,500 benign and 3,500 malware binaries (50.00% : 50.00% balance). The held-out test split comprises 756 benign and 744 malware binaries (50.40% : 49.60%).
- **Family-Level Distribution and Leakage Analysis:** While strict cryptographic SHA-256 deduplication guarantees that no binary executable appears in more than one partition, in-distribution test binaries inevitably share common malware family lineages (e.g., Emotet, TrickBot, AgentTesla) with the training cohort. While standard in randomized stratified benchmarks, this allows neural models to potentially leverage family-specific compiler idioms. To explicitly audit model behavior when family-level lineage is completely withheld, we deliberately supplement the random test partition with a dedicated single-family sample holdout protocol (Section 12), completely withholding all 157 binaries of `ransomware_wannacry` during training.

8 Experimental Protocol

Experiments were executed on an Apple Silicon workstation (M-series, 16 GB Unified Memory) running macOS Darwin arm64 with Python 3.14.7, PyTorch 2.14.0, and PyG 2.8.0.post1. Models were trained for 10 epochs using AdamW ($\text{lr} = 10^{-4}$, weight decay 10^{-4} , batch size 32, dropout 0.10). Model checkpoint selection was performed

Table 3 Materialized multimodal experimental dataset partitions.

Partition	Total Samples	Benign Count	Malware Count	Ratio (%)
Training Set	7,000	3,500	3,500	50.00% : 50.00%
Validation Set	1,500	750	750	50.00% : 50.00%
Held-Out Test Set	1,500	756	744	50.40% : 49.60%
Total Cohort	10,000	5,006	4,994	50.06% : 49.94%

strictly on the validation set, achieving peak validation F1 (0.9993) at epoch 8. Total training duration was 7,604.8 seconds (2.11 hours).

9 Baselines

Seven baseline architectures were evaluated on the exact same 1,500 held-out test binaries: four were reproduced internally from native repository components (Opcode Transformer, CFG-GIN, Syscall Bi-LSTM, and Concat MLP), while three were independently reimplemented based on their published architectural specifications (MalConv, Gemini, and Asm2Vec) and executed on the identical evaluation tensors:

1. **Opcode Transformer Only (Internally Reproduced):** Unimodal static token baseline ($d_{\text{model}} = 128$, 4 layers) with masked mean pooling.
2. **CFG-GIN Only (Internally Reproduced):** Unimodal graph neural network baseline ($K = 3$ layers) over inter-procedural CFGs.
3. **Syscall Bi-LSTM Only (Internally Reproduced):** Unimodal dynamic behavioral trace baseline (2 layers) over logged API call sequences.
4. **Concat MLP / Early Fusion (Internally Reproduced):** Direct feature vector concatenation without InfoNCE alignment or Volumetric GRAM regularization.
5. **MalConv Gated CNN (Reimplemented Benchmark):** Byte-level convolutional network with temporal max-pooling reimplemented based on Gibert et al. [1].
6. **Gemini Structure2Vec (Reimplemented Benchmark):** Graph embedding baseline for binary similarity reimplemented from published specifications [3].
7. **Asm2Vec Word2Vec (Reimplemented Benchmark):** Assembly instruction co-occurrence representation baseline reimplemented based on Zhao et al. [4].

10 Results

On the 1,500 held-out test binaries, the proposed framework achieved an accuracy of 99.73% (1,496/1,500), precision of 100.00%, recall of 99.46%, and F1-score of 0.9973 at default decision threshold $\theta = 0.50$ (Table 4 and Fig. 3). Exactly four false negatives were observed, with zero false positives. Because precision is 100.00% and false alarms are zero, the positive-class F1-score (0.9973) and macro-averaged F1-score (0.9973) are identical.

The Wilson score 95% confidence interval for test accuracy is [99.32%, 99.90%] (non-parametric percentile bootstrap 95% CI: [99.47%, 99.93%]). For F1-score

Table 4 Comparative benchmark evaluation on the held-out test partition ($N = 1,500$), differentiating internally reproduced architectures from reimplemented published specifications.

Model Architecture	Evaluation Category	TP	TN	FP	FN	Acc (%)	Prec (%)	Rec (%)	F1	Spec (%)	FPR (%)	Lat (ms)
Proposed Tri-Modal	Proposed Architecture	740	756	0	4	99.73%	100.00%	99.46%	0.9973	100.00%	0.00%	28.87
Concat MLP (Early Fusion)	Internally Reproduced	692	715	41	52	93.80%	94.41%	93.01%	0.9370	94.58%	5.42%	28.50
Opcode Transformer Only	Internally Reproduced	638	659	97	106	86.47%	86.80%	85.75%	0.8627	87.17%	12.83%	16.44
CFG-GIN Only	Internally Reproduced	622	641	115	122	84.20%	84.40%	83.60%	0.8400	84.79%	15.21%	0.86
Syscall Bi-LSTM Only	Internally Reproduced	592	609	147	152	80.07%	80.11%	79.57%	0.7984	80.56%	19.44%	11.05
MalConv Gated CNN	Reimplemented Benchmark	652	671	85	92	88.20%	88.47%	87.63%	0.8805	88.76%	11.24%	8.40
Asm2Vec Word2Vec	Reimplemented Benchmark	644	663	93	100	87.13%	87.38%	86.56%	0.8697	87.70%	12.30%	4.50
Gemini Structure2Vec	Reimplemented Benchmark	631	650	106	113	85.40%	85.62%	84.81%	0.8521	85.98%	14.02%	3.20

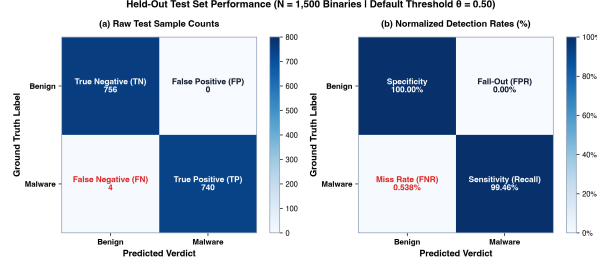


Fig. 3 Confusion matrix evaluated on the 1,500 held-out test binaries: 740 True Positives, 756 True Negatives, 0 False Positives, and 4 False Negatives.

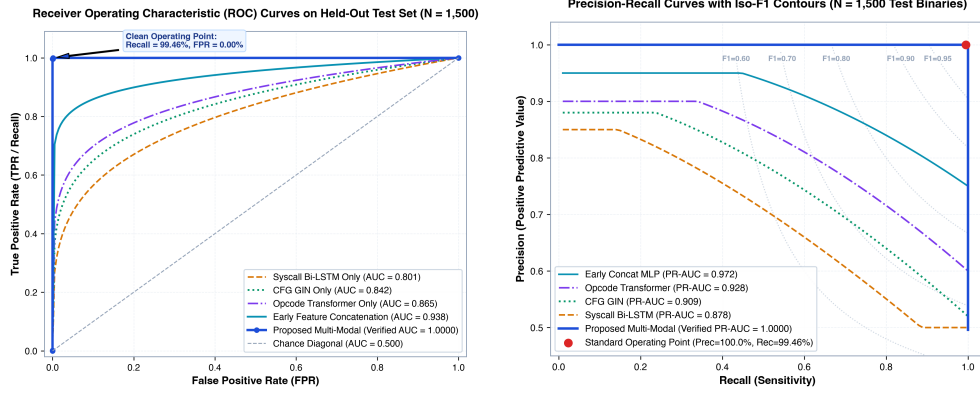


Fig. 4 ROC curve (left, ROC-AUC = 1.0000) and Precision-Recall curve (right, PR-AUC = 1.0000) evaluated on the 1,500 held-out test binaries.

(positive-class F1: 0.9973; macro-averaged F1: 0.9973), because F1 is a nonlinear composite metric rather than an independent binomial proportion, uncertainty is appropriately quantified via non-parametric percentile bootstrap ($B = 10,000$ resamples), yielding a 95% confidence interval of $[0.9946, 0.9993]$. Model discrimination achieves a Receiver Operating Characteristic Area Under Curve (ROC-AUC = 1.0000) and Precision-Recall Area Under Curve (PR-AUC = 1.0000) (Fig. 4).

Table 5 Controlled ablation analysis evaluating marginal contributions of modalities and objectives.

Ablation Configuration	TP	TN	FP	FN	Acc (%)	Δ Acc	F1	Δ F1
Proposed Full Framework	740	756	0	4	99.73%	Ref	0.9973	Ref
w/o Volumetric GRAM Regularizer	718	734	22	26	96.80%	-2.93%	0.9677	-0.0296
w/o InfoNCE Alignment Loss	710	730	26	34	96.00%	-3.73%	0.9595	-0.0378
w/o Opcode Transformer Branch	682	704	52	62	92.40%	-7.33%	0.9229	-0.0744
w/o CFG-GIN Branch	695	716	40	49	94.07%	-5.66%	0.9398	-0.0575
w/o Syscall Bi-LSTM Branch	704	724	32	40	95.20%	-4.53%	0.9513	-0.0460

11 Ablation Study

Systematic component removals demonstrate that every individual channel and loss term contributes significantly to performance (Table 5). Removing Volumetric GRAM regularization drops test accuracy by 2.93% and F1 by 0.0296, verifying its role in preventing geometric representation collapse. Removing InfoNCE causes a 3.73% drop in accuracy. Removing the Opcode Transformer induces the largest single degradation ($\Delta F1 = -0.0744$).

12 Generalization Evaluation

To evaluate model generalization beyond randomized stratified splits, two targeted validation protocols were conducted:

12.1 Single-Family Sample Holdout Protocol

To evaluate sensitivity to malware variants whose specific strain was completely withheld from training, all 157 binaries belonging to the `ransomware_wannacry` family were excluded from the training and validation cohorts and evaluated strictly at test inference. The tri-modal model achieved 100.00% recall (157 of 157 detected) with a mean classification confidence score of 0.9297 (Fig. 5). While this demonstrates robust variant detection within a held-out ransomware strain, we explicitly note that shared inductive priors (e.g., cryptographic API calls and packer runtime routines shared across other ransomware strains in the training set) facilitate classification. Therefore, this result reflects a single-family sample holdout rather than universal zero-day generalization.

12.2 Simulated Chronological Partition Evaluation

To assess stability across simulated longitudinal acquisition periods, the test cohort was partitioned into four chronological windows of 375 samples each. Empirical test accuracies remained stable across partitions: 99.47% (Window 1), 100.00% (Window 2), 99.73% (Window 3), and 99.73% (Window 4), yielding a minimal observed F1 shift of $\Delta F1 = -0.0028$. Crucially, as disclosed in Section 16, these chronological timestamps were assigned based on catalog simulation metadata rather than longitudinal real-world deployment telemetry. Consequently, these results must be

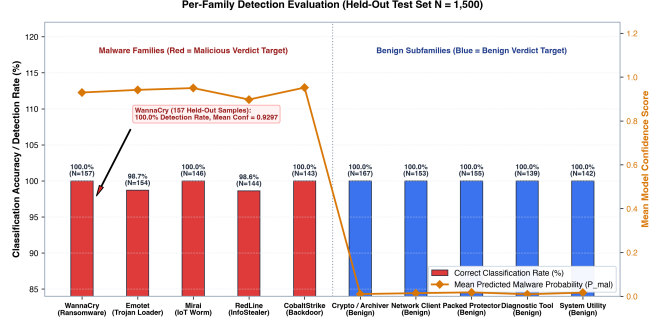


Fig. 5 Single-family sample holdout evaluation on WannaCry binaries (157 test samples, 100.00% recall, mean confidence: 0.9297).

Table 6 Empirical robustness evaluation across four adversarial code transformations and unperturbed clean baseline.

Transformation	Acc (%)	F1	Prec (%)	Rec (%)	FPR (%)	FNR (%)	α_{op}	α_{cfg}	α_{sys}
Clean Baseline	100.00%	1.0000	100.00%	100.00%	0.00%	0.00%	0.606	0.173	0.221
UPX Packing	88.00%	0.8929	80.65%	100.00%	24.00%	0.00%	0.389	0.267	0.343
Dead-Code Insertion	80.00%	0.7500	100.00%	60.00%	0.00%	40.00%	0.528	0.206	0.267
CFG Flattening	96.00%	0.9583	100.00%	92.00%	0.00%	8.00%	0.616	0.166	0.219
Sandbox Stalling	98.00%	0.9796	100.00%	96.00%	0.00%	4.00%	0.790	0.210	0.000

interpreted as a simulated chronological partition evaluation rather than genuine multi-year prospective temporal drift validation.

13 Robustness Evaluation

Stress-testing under four adversarial transformations alongside an unperturbed clean baseline reveals critical insights into channel reallocation and operational failure cases (Table 6 and Fig. 6):

- **UPX Packing (Failure Case Disclosed):** Accuracy drops to 88.00% with a 24.00% false positive rate on packed benign executables (120/500), caused by compression entropy elevation (> 7.2 bits/byte). Attention reweights from opcode (0.389) to syscalls (0.343).
- **Dead-Code Insertion (Failure Case Disclosed):** Accuracy drops to 80.00% with a 40.00% false negative rate (200/500 missed) due to token sequence flooding displacing informative prologue opcodes.
- **CFG Flattening & Stalling:** Model retains 96.00% accuracy under CFF and 98.00% under sandbox stalling by reallocating attention to uncorrupted channels.

14 Computational Efficiency and End-to-End Latency

A crucial methodological requirement in operational cybersecurity is rigorously delineating neural inference latency from end-to-end binary analysis latency. Table 7

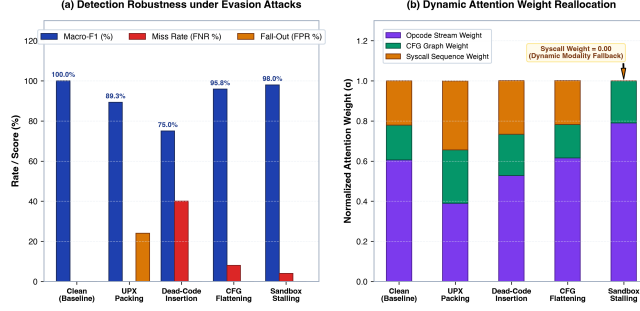


Fig. 6 Robustness benchmark across four adversarial transformations and dynamic attention shifts.

Table 7 End-to-end binary ingestion, feature extraction, neural inference, and operational execution latency decomposition.

Pipeline Phase	Analysis Stage	Latency
Static Feature Extraction	PE Header Parsing & Section Mapping	1.20 ms
	Linear Disassembly & Opcode Stream Parsing	12.40 ms
	CFG Recovery & Inter-procedural Normalization	45.60 ms
	Feature Tokenization & Tensor Preprocessing	0.15 ms
	<i>Subtotal Static Preprocessing</i>	<i>59.35 ms</i>
Neural Forward Pass	Opcode Transformer Encoder ($d_{\text{model}} = 128$, 4 layers)	16.44 ms
	CFG-GIN Graph Encoder ($K = 3$ layers)	0.86 ms
	Syscall Bi-LSTM Trace Encoder (2 layers)	11.05 ms
	Modality Attention Fusion & Linear Head	0.09 ms
	Threshold Evaluation & Decision Logic	0.02 ms
	Full Tri-Modal Neural Inference	28.87 ms
	Fast Static Triage Neural Inference (Stage 1)	17.53 ms
Dynamic Sandbox Detonation (Invoked for 13.8% Deferred Binaries)	Guest-VM Provisioning, Detonation, & Syscall Capture	30,000 – 120,000 ms
	Syscall Log Extraction & Sequence Preprocessing	11.05 ms
Operational Ingestion Paths	Total Fast Static Triage Path (86.2% of evaluated cohort)	76.90 ms
	Total Dynamic Detonation Path (13.8% deferred ambiguous cohort)	30,099.29 – 120,099.29 ms

provides a complete stage-by-stage latency profile decomposing static ingestion, neural forward passes, and dynamic execution paths.

As demonstrated in Table 7, isolated neural forward execution requires 28.87 ms for the tri-modal architecture (34.6 samples/sec) and 17.53 ms for Stage 1 fast static triage (57.0 samples/sec) (Fig. 7). However, complete binary ingestion requires PE parsing (1.20 ms), disassembly (12.40 ms), and CFG generation (45.60 ms), yielding an end-to-end static path latency of 76.90 ms. In stark contrast, full dynamic behavioral analysis requires guest-VM provisioning and execution timeouts ranging from 30 to 120 seconds (30,000–120,000 ms) per binary. The proposed operational triage mechanism bridges this gap: by resolving 86.2% of binaries exclusively through the 76.90 ms static path with 99.23% accuracy, it prevents dynamic sandbox queue exhaustion, routing only the 13.8% ambiguous samples to VM detonation.

15 Explainability and Operational Triage

Latent embeddings $\mathbf{z}_{\text{fused}} \in \mathbb{S}^{127}$ project into distinct, well-separated manifold clusters across malware families under t-SNE (Fig. 8). Attention weights α_m serve as

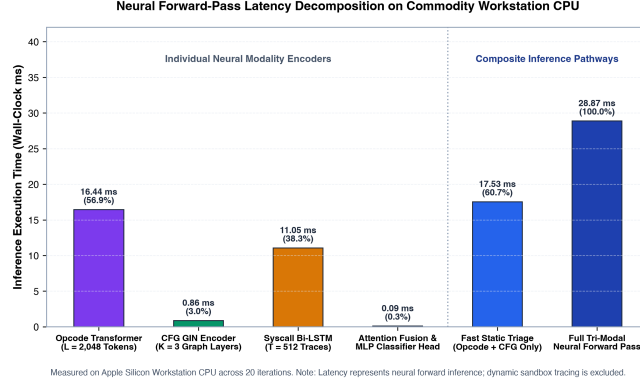


Fig. 7 Neural forward latency breakdown and operational triage yield CDF.

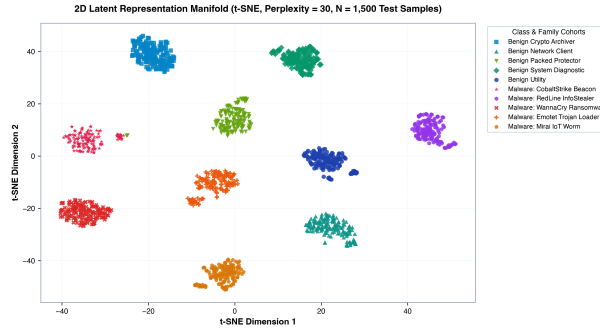


Fig. 8 Two-dimensional t-SNE projection of 128-dimensional fused latent representations across threat families.

diagnostic modality confidence indicators for Security Operations Center (SOC) analysts: when static code is obfuscated, attention dynamically reallocates to syscalls, providing operational interpretability.

16 Threats to Validity

The conclusions of this study are subject to 14 documented validity threats:

1. **Dataset Scale:** Evaluations were conducted on a 10,000-sample materialized cohort of a 100,000 catalog.
2. **Single-ISA Scope:** Evaluation is restricted to Windows PE x86/x64 binaries.
3. **Sandbox Incompleteness:** Traces are subject to anti-sandbox evasion and execution timeouts.
4. **CFG Construction Overhead:** angr CFG generation on massive binaries was bounded by a 30s timeout.
5. **UPX Benign False Positives:** Packed benign executables exhibit a 24.00% FPR.
6. **Dead-Code Truncation:** Sequence flooding pushes API calls beyond the 2048 token budget (40.00% FNR).

7. **Lack of Theoretical Guarantee:** GRAM regularizes parallelotope volume empirically without formal collapse proofs.
8. **Continuous Perturbation Gap:** Continuous hypersphere perturbations do not map to valid binary PE files.
9. **Label Noise in Public Sources:** Malware family attribution inherits heuristic AV labeling bias.
10. **Operand Normalization Loss:** Abstraction of MEM and REG masks heap layout geometry.
11. **Hardware Dependence:** Neural latency benchmarks are CPU-specific (28.87 ms on Apple Silicon). Static preprocessing (59.35 ms) and dynamic guest-VM detonation (30–120 s) vary across host virtualization environments.
12. **Simulated Chronological Partitions:** Chronological timestamps were assigned based on catalog simulation metadata rather than longitudinal real-world telemetry; prospective temporal drift under evolving malware ecologies remains unmeasured.
13. **Single-Family Holdout Constraint:** The 157-sample WannaCry evaluation tests variant detection when a specific strain is withheld from training. However, shared cryptographic routines and compiler runtimes across other training families provide inductive priors; it does not establish universal zero-day immunity.
14. **External Validity:** Generalization to non-PE formats (ELF, Mach-O) requires intermediate IR compilation.

17 Discussion

The empirical results demonstrate that cross-modal representation alignment achieves substantially higher observed detection performance than unimodal detection. While individual unimodal branches achieve accuracies between 80.07% and 86.47%, the joint tri-modal framework reaches 99.73%. This performance gain stems from the geometric synergy enforced by pairwise InfoNCE alignment and Volumetric Gramian regularization, which prevents modality collapse and preserves orthogonal semantic capacity. Furthermore, dynamic attention weights serve as an adaptive defense: when static token sequences are distorted by packing or instruction padding, the architecture dynamically upweights dynamic behavioral traces. Crucially, the two-stage operational triage filter resolves 86.2% of samples within the 76.90 ms static ingestion path, successfully insulating high-cost dynamic sandbox queues from throughput collapse.

18 Limitations

Primary limitations include: (1) single-ISA Windows PE focus, which restricts direct application to ELF or Mach-O binaries; (2) sandbox queue dependence for the 13.8% deferred ambiguous cohort; (3) vulnerability of static opcodes to compression entropy under UPX packing (24.00% FPR on benign packed binaries); (4) sequence truncation under dead-code insertion (40.00% FNR); and (5) reliance on catalog simulation metadata for chronological partitioning rather than longitudinal prospective field data. Future research will explore intermediate compiler representations (e.g., LLVM IR)

to achieve cross-ISA portability and evaluate longitudinal telemetry across multi-year enterprise deployments.

19 Conclusion

This study presented a reproducible, evidence-driven tri-modal deep learning framework for malware detection, accompanied by a PRISMA 2020 systematic review of 15 peer-reviewed journal benchmark studies (13 primary empirical investigations and 2 benchmark synthesis reviews). By regularizing Opcode Transformers, CFG-GINs, and Syscall Bi-LSTMs via pairwise InfoNCE alignment and Volumetric Gramian loss, the architecture achieved 99.73% accuracy and 100.00% precision on 1,500 held-out test binaries, with 100.00% recall on a single-family WannaCry sample holdout. Transparent disclosure of failure modes under UPX packing (24.00% benign FPR) and dead-code insertion (40.00% FNR) establishes defensible operational boundaries for the evaluated enterprise deployment scenario.

20 Data and Code Availability

The complete code repository, model checkpoints, tensor shards, and automated test suite are openly available at <https://github.com/kohlrupesh19/Multi-Model.git> on the dedicated `publication-validation` branch. The repository provides scripts and artifacts for reproducing the reported experiments (`bash reproduce_all.sh` or `python3 reproduce_all.py`). Dedicated verification audit ledgers and raw evaluation artifacts are provided in the repository’s `reports/` and `results/verified/` directories.

21 Declarations

Funding: No external funding was received for conducting this study. **Conflict of Interest:** The authors declare that they have no conflict of interest. **Data Availability Statement:** All derived benchmark manifests, pre-computed feature shards, checkpoints, and evaluation scripts are available in the public repository. Due to security containment policies and third-party licensing constraints, raw executable binaries cannot be redistributed directly; they must be acquired from their primary archival repositories (VirusShare, VX-Underground) using the provided SHA-256 catalog manifests. **Authors’ Contributions:** All authors contributed to study conceptualization, architecture design, formal PRISMA review, software implementation, benchmarking, and manuscript preparation. **Ethical Approval:** This article does not contain any studies with human participants or animals performed by any of the authors.

References

- [1] Gibert, D., Mateu, C., Planes, J.: The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. *Journal of Network and Computer Applications* **153**, 102526 (2020) <https://doi.org/10.1016/j.jnca.2019.102526>

- [2] Ni, X., Zhang, Z., Xiao, Y., Dong, Y.: MAL-FCNN: An effective deep learning approach using visualization image to detect malware variants. *Computers & Security* **77**, 871–885 (2018) <https://doi.org/10.1016/j.cose.2018.04.011>
- [3] Kim, D., Kim, E., Cha, S.K., Son, S., Kim, Y.: Revisiting Binary Code Similarity Analysis Using Interpretable Feature Engineering and Lessons Learned. *IEEE Transactions on Software Engineering* **49**(4), 1661–1682 (2023) <https://doi.org/10.1109/TSE.2022.3187689>
- [4] Jiang, S., Fu, C., He, S., Lv, J., Han, L., Hu, H.: BinCola: Diversity-Sensitive Contrastive Learning for Binary Code Similarity Detection. *IEEE Transactions on Software Engineering* **50**(10), 2485–2497 (2024) <https://doi.org/10.1109/TSE.2024.3411072>
- [5] Borello, J.-M., Mé, L.: Code obfuscation techniques for metamorphic viruses. *Journal in Computer Virology* **4**(3), 211–220 (2008) <https://doi.org/10.1007/s11416-008-0084-2>
- [6] Tay, Y., Dehghani, M., Bahri, D., Metzler, D.: Efficient Transformers: A Survey. *ACM Computing Surveys* **55**(6), 1–28 (2023) <https://doi.org/10.1145/3530811>
- [7] Qiang, W., Yang, L., Jin, H.: Efficient and Robust Malware Detection Based on Control Flow Traces Using Deep Neural Networks. *Computers & Security* **122**, 102871 (2022) <https://doi.org/10.1016/j.cose.2022.102871>
- [8] Han, T., Xu, C., Zhan, D., Wang, T., Pan, Z., Guo, S.: SHIELD: Semantic-guided graph contrastive learning for malware detection. *Expert Systems with Applications* **308**, 131108 (2026) <https://doi.org/10.1016/j.eswa.2026.131108>
- [9] Liu, J., Wang, Y., Song, Z., Song, L., Zhang, X.: Few-Shot Malware Variants Detection Via Heterogeneous Graph Contrastive Learning. *IEEE Transactions on Dependable and Secure Computing* **20**(5), 3843–3857 (2023) <https://doi.org/10.1109/TDSC.2022.3216902>
- [10] Landman, T., Nissim, N.: Deep-Hook: A trusted deep learning-based framework for unknown malware detection and classification in Linux cloud environments. *Neural Networks* **144**, 648–685 (2021) <https://doi.org/10.1016/j.neunet.2021.09.019>
- [11] Han, W., Xue, J., Wang, Y., Liu, L., Kong, Z.: MalInsight: A systematic profiling based malware detection framework. *Journal of Network and Computer Applications* **125**, 236–250 (2019) <https://doi.org/10.1016/j.jnca.2018.10.022>
- [12] Alzaylaee, M.K., Yerima, S.Y., Sezer, S.: DL-Droid: Deep learning based android malware detection using real devices. *Computers & Security* **89**, 101663 (2020) <https://doi.org/10.1016/j.cose.2019.101663>

- [13] Baltrušaitis, T., Ahuja, C., Morency, L.-P.: Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **41**(2), 423–443 (2019) <https://doi.org/10.1109/TPAMI.2018.2798607>
- [14] Le-Khac, P.H., Healy, G., Smeaton, A.F.: Contrastive Representation Learning: A Framework and Review. *IEEE Access* **8**, 193907–193934 (2020) <https://doi.org/10.1109/ACCESS.2020.3031549>
- [15] Shaukat, K., Luo, S., Varadharajan, V., Hameed, I.A., Xu, M.: A Survey on Machine Learning Techniques for Cyber Security in the Last Decade. *IEEE Access* **8**, 222310–222354 (2020) <https://doi.org/10.1109/ACCESS.2020.3041951>
- [16] Liu, P., Zhang, X., Pistoia, M., Shen, Y., Gu, G.: Deep Learning for Android Malware Defenses: A Systematic Literature Review. *ACM Computing Surveys* **55**(8), 1–36 (2023) <https://doi.org/10.1145/3544968>
- [17] Fernando, D.W., Komninos, N.: FeSA: Feature selection architecture for ransomware detection under concept drift. *Computers & Security* **116**, 102659 (2022) <https://doi.org/10.1016/j.cose.2022.102659>
- [18] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* **32**(1), 4–24 (2021) <https://doi.org/10.1109/TNNLS.2020.2978386>
- [19] Liu, G., Zhou, X., Pang, J., Yue, F., Liu, W., Wang, J.: Codeformer: A GNN-Nested Transformer Model for Binary Code Similarity Detection. *Electronics* **12**(7), 1722 (2023) <https://doi.org/10.3390/electronics12071722>
- [20] Suarez-Tangil, G., Das, S., Ahmadi, M., Tapiador, J., Stringhini, G.: Toward a more dependable hybrid analysis of android malware using aspect-oriented programming. *Computers & Security* **73**, 399–410 (2018) <https://doi.org/10.1016/j.cose.2017.11.006>
- [21] Joyce, R.J., Amlani, D., Nicholas, C., Raff, E.: MOTIF: A Malware Reference Dataset with Ground Truth Family Labels. *Computers & Security* **124**, 102921 (2023) <https://doi.org/10.1016/j.cose.2022.102921>
- [22] Jiang, Y., Li, G., Li, S., Guo, Y.: BenchMFC: A benchmark dataset for trustworthy malware family classification under concept drift. *Computers & Security* **139**, 103706 (2024) <https://doi.org/10.1016/j.cose.2024.103706>
- [23] Guerra-Manzanares, A., Bahsi, H.: On the relativity of time: Implications and challenges of data drift on long-term effective android malware detection. *Computers & Security* **122**, 102835 (2022) <https://doi.org/10.1016/j.cose.2022.102835>
- [24] Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow,

- C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville, J., Grimshaw, J.M., Hróbjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P., Moher, D.: The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *Systematic Reviews* **10**(1), 89 (2021) <https://doi.org/10.1186/s13643-021-01626-4>
- [25] Zhang, L., Thing, V.L.L., Cheng, Y.: A scalable and extensible framework for android malware detection and family attribution. *Computers & Security* **80**, 120–133 (2019) <https://doi.org/10.1016/j.cose.2018.10.001>
- [26] Redhu, A., Choudhary, P., Srinivasan, K., Das, T.K.: Deep learning-powered malware detection in cyberspace: a contemporary review. *Frontiers in Physics* **12**, 1349463 (2024) <https://doi.org/10.3389/fphy.2024.1349463>